

L2-3.4. Elastic: Using Logstash

Task 1 - Introduction

Elastic's [Logstash](#) (opens in new tab) is an open-source data processing engine that allows you to collect, enrich, and transform data from different sources. Tools in the Elastic Stack are Logstash, Elasticsearch and Kibana, to create a complete data processing and visualization pipeline.

Task 2 - Installing Logstash

The Kibana interface can be accessed over port 5601 or via the Elastic shortcut on your desktop. Head over to the `/Downloads` folder, switch to the `root` user, and use `dpkg` to install Logstash. In a production environment, you would typically download this file using `wget` or `curl`, or by adding the Elastic APT repository to ensure you are installing the latest release.

```
ubuntu@logstash:~$ cd Downloads
ubuntu@logstash:~/Downloads$ ls
logstash.deb
ubuntu@logstash:~/Downloads$ sudo su
root@logstash:/home/ubuntu/Downloads# dpkg -i logstash.deb
Selecting previously unselected package logstash.
(Reading database ... 522226 files and directories currently in
stalled.)
Preparing to unpack logstash.deb ...
Unpacking logstash (1:9.2.4-1) ...
Setting up logstash (1:9.2.4-1) ...
```

Starting Logstash:

```
root@logstash:/home/ubuntu/Downloads# systemctl start logstash
root@logstash:/home/ubuntu/Downloads# systemctl enable logstash
root@logstash:/home/ubuntu/Downloads# systemctl status logstash
● logstash.service - logstash
   Loaded: loaded (/usr/lib/systemd/system/logstash.service; e
nabled; preset: enabled)
   Active: active (running) since Tue 2026-02-24 05:24:43 UTC;
```

10s ago

. . .

Elastic Data Collection Tools

Elastic Stack Component	Primary Role	Capability	Management	Common Use Case
Logstash	Data processing pipeline	Advanced filtering, parsing, and enrichment	Standalone configuration files	Custom pipelines and complex transformations
Beats	Lightweight data shippers	Minimal processing	Individual Beat configuration files	Sending logs and metrics from endpoints
Elastic Agent	Unified data collection agent	Basic with Agent integrations	Centrally managed via Fleet Server in Kibana	Modern, centralized Elastic deployments

Answer the questions below

Which Elastic stack component offers advanced filtering, parsing, and enrichment of log data?

Logstash

Which Logstash version number did you just install?

9.2.4

Task 3 - How Logstash Works

Logstash is a server-side data processing engine that ingests data from a variety of sources, transforms and enriches it, and then forwards it to one or more destinations. It relies entirely on a modular plugin architecture, which makes it highly flexible.

These [plugins](#) are organized into three main categories:

- **Input plugins:** Collect data from sources such as files, network ports, Beats agents, or APIs

- **Filter plugins:** Parse, normalize, enrich, and manipulate events as they pass through the pipeline
- **Output plugins:** Send the processed data to destinations such as , a file, or another service

Elasticsearch

A Logstash configuration file is structured around these three components: `input` , `filter` , and `output` . Together, they define how data flows through the pipeline.

Inputs: Logstash provides a wide range of [input plugins\(opens in new tab\)](#) that allow you to ingest data from diverse sources including log files, system metrics, databases, message queues, APIs, and more. These input plugins facilitate data collection in various formats and protocols. Below are a few commonly used Logstash input plugins.

Plugin Name	Description
File	Reads data from files in real-time or as a batch. Useful for ingesting log files or other structured data stored on the local file system
Beats	Ingests data from Beats shippers (such as Filebeat or Metricbeat), lightweight agents designed to ship various data types to Logstash
TCP	Listens for incoming TCP connections and reads data from them. Useful for receiving data from network devices or other systems over TCP
UDP	Listens for incoming UDP packets and reads data from them. Ideal for receiving log data or other types of data sent over UDP
HTTP	Acts as a web server and reads data from HTTP requests. Useful for receiving data from webhooks, REST APIs, or other HTTP-based sources

Filter: Once the data is ingested, Logstash provides a rich set of [filter plugins\(opens in new tab\)](#) that enable you to manipulate and transform the data. Filters allow you to parse, enrich, modify, and filter incoming data.

Plugin Name	Description
Grok	Parses unstructured log data using custom patterns and extracts structured fields from it

Plugin Name	Description
Mutate	Performs various mutations on event fields, such as renaming, removing, converting data types, and more
Date	Parses and manipulates dates and timestamps in event fields. Allows you to extract, format, or convert timestamps to a desired configuration
Translate	Translates values in event fields based on defined mappings. It can be used for data normalization or mapping codes to meaningful values
Prune	Removes fields from events using a whitelist or blacklist based on field names or field values

Output: After the data is filtered, Logstash provides [output plugins](#)(opens in new tab) to send the transformed data to different destinations. These destinations can include Elasticsearch for indexing and search, various databases, message queues, cloud storage services, monitoring systems, and more. Some of the common output plugins are explained below.

Plugin Name	Description
Elasticsearch	Sends events to Elasticsearch, a popular search and analytics engine. Allows you to index and store data for further analysis and search
File	Writes events to files on the local file system or a network-mounted file system. Useful for storing data locally or archiving log files
HTTP	Sends events to an external service over HTTP or HTTPS. Useful for integrating Logstash with REST APIs, webhooks, or SOAR platforms
STDOUT	Prints events to the console (standard output). Useful for debugging or quick data inspection
Kafka	Produces messages to an Apache Kafka message broker. Enables Logstash to send data to other systems via Kafka

Answer the questions below

Which plugin parses unstructured log data using custom patterns?

Grok

Which filter plugin is used to perform mutation of event fields?

Mutate

Task 4 - Configuring Logstash

In this task, we will create a simple Logstash pipeline using an input, filter, and output plugin. Imagine you want to receive JSON-formatted log data over TCP on port `5456` and forward it to Elasticsearch. Let's configure the pipeline step by step. Logstash pipeline configuration files are typically stored in the `/etc/logstash/conf.d/` directory. Any new pipeline configuration file you create should be placed in this directory.

Applying the Input

We will use the TCP input plugin in Logstash. Reviewing the [documentation](#)(opens in new tab), we see that the only required configuration option is the port Logstash should listen on.

Setting	Input type	Required
dns_reverse_lookup_enabled	boolean	No
ecs_compatibility	string	No
host	string	No
mode	string	No
port	number	Yes
proxy_protocol	boolean	No

Since we want Logstash to listen on port `5456`, the input section of our configuration would be written as shown below. It is important to note that, in Logstash configurations, we use `=>` to define field-value pairs within plugin settings. You can see below that we are also specifying the codec. In Logstash, a codec defines how incoming data is decoded or encoded as it enters or leaves the pipeline. You can think of it as a translator between raw data and structured events. By default, the TCP input treats incoming data as plain text. However, in our scenario, we expect the logs to be sent in JSON format.

```
input {
  tcp {
    port => 12345
    codec => json
  }
}
```

```
}  
}
```

Adding Filter Plugins: As mentioned above, our example input is in JSON format. Looking at the [documentation\(opens in new tab\)](#), we can see that only the `source` is required. Let's set it up.

Setting	Input type	Required
ecs_compatibility	string	No
skip_on_invalid_json	boolean	No
source	string	Yes
tag_on_failure	array	No
target	string	No

Required Configuration Option

The following configuration setting allows Logstash to parse JSON-formatted data stored in the `message` field. Instead of treating the incoming data as a plain text string, the `json` filter reads the contents of the `message` field, extracts the key-value pairs within it, and converts them into structured fields. This ensures the data can be properly indexed, searched, and analyzed in Elasticsearch.

```
filter {  
  json {  
    source => "message"  
  }  
}
```

Configuring the Output: Finally, we want to send our data to Elasticsearch. The [documentation\(opens in new tab\)](#) indicates that no settings are strictly mandatory for the Elasticsearch output plugin. However, in practice, it is common to define fields such as `hosts` and `index` to explicitly specify the Elasticsearch instance and the index where the data should be stored. This provides clearer control over where events are sent and how they are organized.

```

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "your_index_name"
  }
}

```

Your Completed Configuration: We now have a simple configuration file that demonstrates the use of input, filter, and output plugins in Logstash. If you wanted to use this, or any configuration file, you'd save it as `configname.conf` in the `/etc/logstash/conf.d/` directory.

```

input {
  tcp {
    port => 5456
  }
}

filter {
  json {
    source => "message"
  }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "your_index_name"
  }
}

```

Answer the questions below

Check out the File input plugin documentation (opens in new tab). Which configuration option is required?

path

Which field does the JSON filter plugin read the contents of?

message

Task 5 - Configuring Inputs

File Input Plugin: The [File input plugin \(opens in new tab\)](#) is one of the most commonly used Logstash inputs. It allows Logstash to monitor and ingest data from files on a local system or network share.

- `path`: Specifies the path to the file(s) to be read
- `start_position`: Defines where Logstash should start reading the file. In this example, `beginning` means it will start reading from the beginning of the file
- `sincedb_path`: Sets the path to the `sincedb` file, which keeps track of the current position of the file read

```
input {  
  file {  
    path => "/path/to/your/file.log"  
    start_position => "beginning"  
    sincedb_path => "/dev/null"  
  }  
}
```

Beats Input Plugin: The [Beats input plugin \(opens in new tab\)](#) allows Logstash to receive data from Beats, lightweight data shippers that send logs, metrics, and network data to the Elastic Stack. This is commonly used for Filebeat, Metricbeat, Winlogbeat, and others.

- `port`: Specifies the port number on which Logstash should listen for Beats connections

```
input {  
  beats {  
    port => 5044  
  }  
}
```



```
}  
}
```

TCP Input Plugin: The [TCP input plugin\(opens in new tab\)](#) listens for log data sent over TCP connections. Useful for ingesting logs from custom applications, network devices, or any service that can push data via TCP.

- `port`: Sets the TCP port on which Logstash should listen for incoming data
- `codec`: tells Logstash how to interpret incoming data. Common options are `plain` (default) or `json` if sending JSON formatted logs

```
input {  
  tcp {  
    port => 5000  
    codec => json  
  }  
}
```

UDP Input Plugin: The [UDP input plugin\(opens in new tab\)](#) listens for log data sent over UDP connections. Commonly used for network devices, syslog servers, or lightweight shippers that prefer UDP for low-overhead transmission.

- `port`: Sets the UDP port on which Logstash should listen for incoming data
- `codec`: Specifies the codec to be used for decoding the incoming data. In this example, the data is treated as plain text

```
input {  
  tcp {  
    port => 514  
    codec => "plain"  
  }  
}
```

HTTP Input Plugin: The [HTTP input plugin\(opens in new tab\)](#) creates an HTTP endpoint that allows external systems, applications, or webhooks to send data directly to Logstash. It's commonly used for ingesting JSON payloads, web application logs, or custom API data.

- `port`: Specifies the HTTP port on which Logstash should listen for incoming HTTP requests

```
input {  
  http {  
    port => 8080  
  }  
}
```

Each input plugin provides specific configuration options tailored to its functionality and requirements. The examples above demonstrate how to configure the plugins by specifying necessary settings, such as file paths, ports, and codecs.

Answer the questions below

Imagine you want to ingest native Linux syslog into your Elastic instance. Which Logstash input plugin would you use to monitor and ingest the locally hosted syslog file?

File

Check out the documentation (opens in new tab) for the Google Cloud Storage input plugin.

What is the one required input configuration setting?

bucket_id

Task 6 - Filter Configurations

Mutate Filter: The Mutate filter(opens in new tab) allows you to perform mutations on fields, such as renaming or replacing fields within your events. There are several mutations available.

This `add_field` configuration uses the mutate filter to add a new field called `new_field` with the value `new_value` to each event. The mutate filter provides various operations for modifying fields.

```
filter {  
  mutate {  
    add_field => { "new_field" => "new_value" }  
  }  
}
```

```
}  
}
```

The `rename` configuration utilizes the mutate filter to rename a field. In the example below, the field name `IPAddress` is renamed to `client_ip`.

```
filter {  
  mutate {  
    rename => { "IPAddress" => "client_ip" }  
  }  
}
```

Grok Filter: The Logstash [Grok filter \(opens in new tab\)](#) parses unstructured text logs into structured fields, making them easier to analyze in Kibana. Think of it as pattern matching for your logs. The configuration below uses the grok filter to extract data from the `message` field based on a predefined pattern (`%{PATTERN}`). The extracted data will be stored in a new field called `field_name`.

```
filter {  
  grok {  
    match => { "message" => "%{PATTERN:field_name}" }  
  }  
}
```

Pruning Fields: The [Prune filter \(opens in new tab\)](#) allows you to remove fields from your log data based on a whitelist or blacklist of field names. This configuration utilizes the prune filter to remove empty fields from the event, except for those specified in the `whitelist_names` parameter. Only fields `field1` and `field2` will be retained in the event.

```
filter {  
  prune {  
    whitelist_names => ["field1", "field2"]  
  }  
}
```

Dropping Events: The [Drop filter\(opens in new tab\)](#) is used to remove an event from the pipeline. Once an event is dropped, it will not be sent to the output. Check out the example below in which we discard any event where the `status` field equals `"error"`.

```
filter {
  if [status] == "error" {
    drop { }
  }
}
```

Parsing Key-Value Pairs: The [KV filter\(opens in new tab\)](#) turns text with key-value pairs into separate fields. It splits each pair using `&` and separates the key from the value using `=`. For example, if we used the configuration below, we could transform the log line `user=alice&status=active&session=12345®ion=west` into separate fields.

- user `alice`
- status `active`
- session `12345`
- region `west`

```
filter {
  kv {
    field_split => "&"    # separates each key-value pair
    value_split => "="    # separates key from value
  }
}
```

Answer the questions below

| Which filter plugin is used to rename, replace, and modify event fields?

mutate

| Which filter plugin is used to remove fields from events?

prune

Change the mutate filter configuration in the example below to rename the field `src_ip` to `source_ip`.

```
filter {  
  mutate {  
    # Provide the updated line below as your answer  
    configuration_name => { "from_field" => "to_field" }  
  }  
}  
rename => { "src_ip" => "source_ip" }
```

Task 7 - Defining the Destination

Logstash output plugins determine where processed events are sent. Each plugin serves a specific purpose, such as sending data to Elasticsearch, writing to a file, or forwarding to another server.

Elasticsearch Output: The [Elasticsearch output plugin \(opens in new tab\)](#) sends events to an Elasticsearch cluster or [data stream \(opens in new tab\)](#) for indexing. Here is an example configuration that uses `data_stream => true`, which sends events to a data stream rather than a single static index. This is useful for efficiently ingesting logs, metrics, or security events.

```
output {  
  elasticsearch {  
    hosts => ["hostname:9200"]  
    data_stream => true  
  }  
}
```

File Output: The [File output plugin \(opens in new tab\)](#) writes events to a local file. Each event is appended as a new line.

```
output {  
  file {  
    path => "/path/to/output.txt"  
  }  
}
```

Logstash Output: The [Logstash output plugin \(opens in new tab\)](#) forwards events to another Logstash instance for further processing. The `host` parameter specifies the destination Logstash server's address, and the `port` parameter defines the port to which the events will be sent.

```
output {
  logstash {
    host => "destination_host"
    port => 5000
  }
}
```

TCP Output: The [TCP output plugin \(opens in new tab\)](#) sends events to a remote TCP server. Again, the `host` parameter specifies the destination server's address, and the `port` parameter defines the port to which the events will be sent.

```
output {
  tcp {
    host => "destination_host"
    port => 5000
  }
}
```

RabbitMQ Output: The [RabbitMQ output plugin \(opens in new tab\)](#) publishes events to a RabbitMQ server. In this example, the `host` parameter specifies the RabbitMQ server's address, while `exchange` and `routing_key` define the exchange and routing key for message routing within RabbitMQ.

```
output {
  rabbitmq {
    host => "localhost"
    exchange => "my_exchange"
    routing_key => "my_routing_key"
  }
}
```

Syslog Output: The [Syslog output plugin](#)(opens in new tab) sends events to a Syslog server over TCP or UDP. The `host` parameter specifies the Syslog server's address, the `port` parameter defines the port to which the events will be sent, and the `protocol` parameter specifies the transport protocol (in this case, UDP).

```
output {
  syslog {
    host => "syslog_server"
    port => 514
    protocol => "udp"
  }
}
```

Stdout Output: The [Stdout output plugin](#)(opens in new tab) prints events to the console. It is primarily used for debugging purposes to visually inspect the processed events and verify the Logstash pipeline's transformations.

```
output {
  stdout { }
}
```

Answer the questions below

stdout

Check out the sample syslog output configuration below. What are the values of setting1, setting2, and setting3 to complete the configuration?

```
output {
  syslog {
    setting1 ⇒ "my_host.com"
    setting2 ⇒ 514
    setting3 ⇒ "udp"
  }
}
host, port, protocol
```

Task 8 - Putting Pipelines Into Practice

Configuring Auth Logs: Authentication logs are native Linux system logs that record security-relevant events related to user access and privilege usage. On Ubuntu systems, these events are written to `/var/log/auth.log` and include SSH login attempts, failed authentication attempts, sudo activity, and other access control events. Because they capture both successful and failed login behavior, authentication logs are a critical source of information.

Input Configuration: Using your favorite text editor, go ahead and ensure you're running as `root` and create a new configuration file named `auth.conf` in `/etc/logstash/conf.d`. For this first portion of the configuration, we instruct Logstash to monitor and ingest data from the system's authentication log located at `/var/log/auth.log`. The `start_position => "beginning"` setting ensures that Logstash reads the file from the start the first time it runs. The `sourcedb_path => "/dev/null"` option disables Logstash's default file tracking mechanism.

```
input {
  file {
    path => "/var/log/auth.log"
    start_position => "beginning"
    sourcedb_path => "/dev/null"
  }
}
```

Linux authentication logs follow a predictable syslog format, but by default, each entry is stored as a single unstructured `message` field. The `grok` filter allows us to match patterns within that message and extract specific components, such as the timestamp at the beginning of the log entry. Once extracted, the `date` filter converts that timestamp into Logstash's standard `@timestamp` field. Go ahead and add the following lines to your configuration file below the input configuration.

```
filter {
  grok {
    match => {
      "message" => "%{TIMESTAMP_ISO8601:timestamp} %{NOTSPACE:[host][name]} %{WORD:program}(?:\\[%{NUMBER:pid}\\])?: %{GREEDYDATA:msg}"
    }
  }
}
```



```

    }
    date {
      match => ["timestamp", "ISO8601"]
      target => "@timestamp"
    }
  }
}

```

Output Configuration: Next, we need to decide where to send our logs. We could output them to a file or use any one of the Logstash output plugins we previously explored, but since we already have Elasticsearch and Kibana running, let's send them there with the following. Go ahead and add this to your configuration file, then save and exit. These lines configure Logstash to send collected logs securely to Elasticsearch, creating a daily index so we can visualize and analyze them in Kibana.

```

output {
  elasticsearch {
    hosts => ["https://localhost:9200"]
    user => "elastic"
    password => "pn00IuML9u43_yKb688y"
    index => "auth-logs-%{+YYYY.MM.dd}"
    ssl_verification_mode => "none"
    ssl_enabled => true
  }
}

```

Your Complete Configuration: Looking at your final `auth.conf` file, it should appear as shown below. To recap, we are using the file plugin to read the `auth.log` file, filtering it with Grok and date, and sending the output to Elasticsearch on port `9200`, where it will appear in our Kibana instance.

```

input {
  file {
    path => "/var/log/auth.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

```

```
}  
}
```

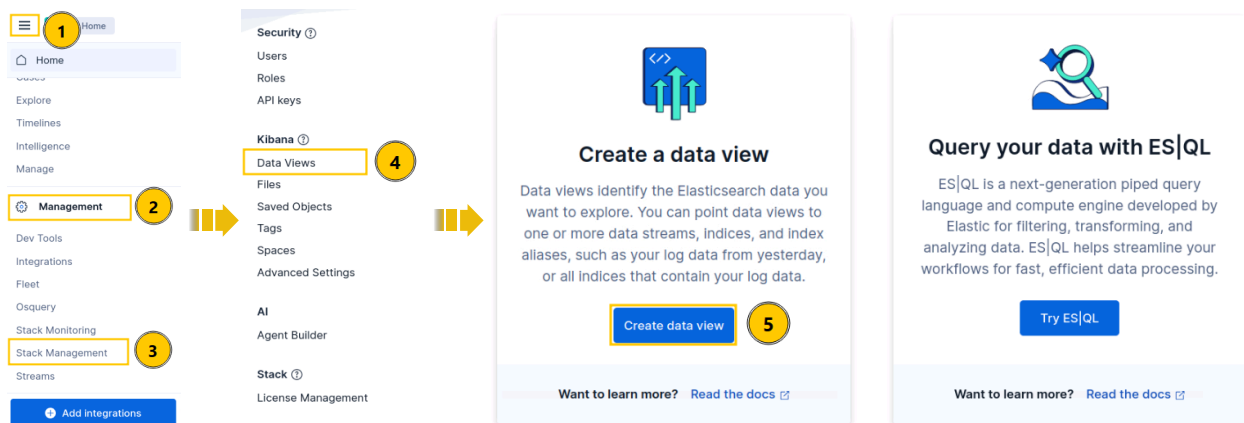
Setting Permissions and Restarting Logstash: Our configuration file is good to go, but before we jump into Kibana, we need to set the proper permissions for the `logstash` user so that it can read the `auth.log` file. Go ahead and run the following commands to set the permissions, restart Logstash, and check its status.

Set Permissions and Restart Logstash

```
root@logstash:/home/ubuntu/Downloads# usermod -aG adm logstash  
root@logstash:/home/ubuntu/Downloads# systemctl restart logstash  
h  
root@logstash:/home/ubuntu/Downloads# systemctl status logstash  
● logstash.service - logstash  
   Loaded: loaded (/usr/lib/systemd/system/logstash.service; e  
nabled; preset:>  
   Active: active (running) since Thu 2026-02-26 05:01:04 UTC;  
9s ago  
   ...
```

Visualizing the Log Data: All of our configuration settings are ready, the permissions have been set, and Logstash has been restarted. The last thing to do is to hop into Kibana and take a look at our logs. Let's first set up a Data view to visualize our new auth log index pattern.

1. Click the Kibana menu
2. Scroll down to **Management**
3. Select **Stack Management**
4. Click **Data Views** under Kibana
5. Choose **Create data view**



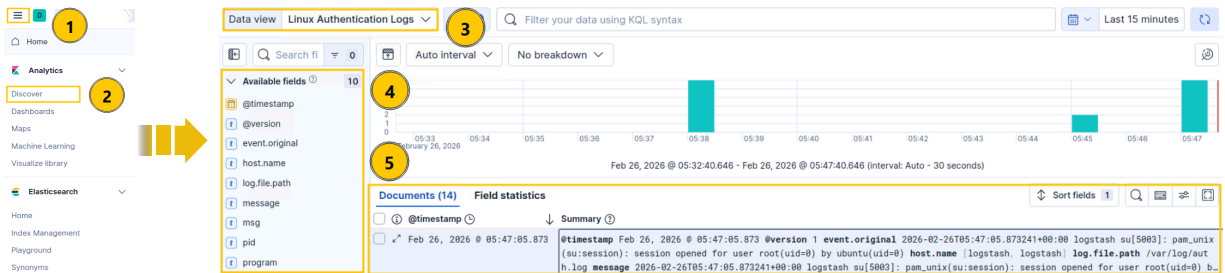
Next, we will name our data view and select the index pattern.

1. Name your data view `Linux Authentication Logs`
2. Enter `auth-logs-*` to match the index pattern we set up in the configuration
3. Select **Save data view to Kibana**

Lastly, let's head to Kibana Discover to confirm our configuration file is set up properly and view our incoming log data.

1. Select the Kibana menu
2. Click Discover

3. Select your **Linux Authentication Logs** data view
4. View your available fields
5. Inspect your incoming event data



Answer the questions below

Which two filters did you use in your auth.conf configuration file?

grok, date

Add a new user via the command line with `useradd -m detective_elastic`. Head back into Kibana to investigate the auth.log events. What is the program field value for the latest events?

useradd

Add a password for the new user using `passwd detective_elastic`. Investigate the password change event in Kibana. What is the full msg field value for the password change event?

`pam_unix(passwd:chauthtok): password changed for detective_elastic`

Task 9 - Conclusion

Learned how to manage the three fundamental components of a Logstash pipeline, input, filter, and output, to transform raw data into searchable events. By building a custom configuration from scratch, you successfully managed the ingestion, filtering, and delivery of native Linux authentication logs to your Elasticsearch instance.